Mid Project Report- Data Mining
Gunn Madan, Harini Mohan
03/27/2026

Dataset source : https://www.ncei.noaa.gov/cdo-web/

Our initiative uses past daily measurements from NOAA Climate Data Online to identify abnormal weather events. A temperature or precipitation observation that substantially departs from the usual seasonal patterns for a particular city is called an anomaly. As of right now, we have finished gathering and preparing data for three locations: Los Angeles, Chicago, and New York City. Subject to station completeness, we combined daily observations for TMAX, TMIN, and PRCP to create a cleaned dataset spanning around 10 years of data per city. The cleaned dataset, which consists of both raw variables and engineering characteristics needed for anomaly identification, is kept in a single file called clean_weather_data.csv.

Additionally, we have implemented the first wave of feature engineering, which includes rolling mean, rolling standard deviation, temperature range, day-of-year, and month extraction. These elements were included to facilitate machine learning-based identification as well as seasonality-aware baselines. To establish seasonal tendencies and make sure the data behaves as predicted across the three cities, preliminary exploratory analysis and visualizations were created.

Results Obtained:

We used seasonal z-scores to establish an initial baseline for anomaly identification. By grouping observations by day of the year, computing the seasonal mean and standard deviation, and then rating each day's variation in relation to that seasonal baseline, this approach creates a seasonal reference for every city. We created anomaly flags for daily maximum temperature (TMAX) using the original threshold setting, and we created city-specific plots that displayed TMAX trends over time with flagged anomalies highlighted.In order to create a machine learning-based anomaly detector, we also put into practice a prototype Isolation Forest model. The model generated anomaly predictions throughout the dataset after being trained on manufactured characteristics. For comparison, the seasonal baseline statistics and anomaly flags (anomaly_z, anomaly_if) from both detection methods were exported as model_output_data.csv.

Regarding early trends, Isolation Forest indicates anomalies differentially based on city and feature interactions, but the seasonal z-score detector flags a reasonable amount of TMAX anomalies across all cities. Although the results are still regarded as preliminary, they demonstrate that the pipeline is operating completely and generating comprehensible anomalous outputs that are appropriate for further analysis in the following stage.

We are on schedule in terms of the anticipated milestones. Both a machine learning detector (Isolation Forest) and a baseline detector (seasonal z-score) have been deployed once data collecting, cleaning, and feature engineering are finished. We have preliminary graphics and working outputs saved to CSV that are immediately applicable to the final report and demo. The finalized time-based train/test experimental setup, the entire evaluation framework (including proxy-label precision and agreement/stability analysis), and the reliable MAD-based detector are the main unfinished pieces.

Future Work:

In order to properly accomplish the project's goals, we will enhance the system and evaluation over the next phase. Because precipitation might be heavy-tailed and sensitive to outliers, we will employ median absolute deviation (MAD) to provide robust anomaly identification. Instead of focusing on just one variable, we will expand the detection pipeline to reliably handle TMIN and PRCP using the same seasonality-aware reasoning used for temperature. In order to maintain temporal structure and avoid leakage during model fitting and threshold selection, we will also finalize time-based train/test divides.

By presenting technique stability over time, agreement among detectors, and accuracy using extreme-percentile proxy labels, we will improve assessment. By comparing high-scoring anomalies to publicly recorded severe weather occurrences like heat waves, cold snaps, and significant precipitation events, we will also offer qualitative confirmation. In order to make the findings evident in both the final report and the demo movie, we will lastly enhance and standardize the visualizations for every city and variable.

Screenshots of current code for cleaning/feature engineering, EDA (a small sample), and initial modeling

Importing original dataset

```
import pandas as pd

df = pd.read_csv('/content/drive/MyDrive/WeatherProject_DataMining/OriginalWeatherDataset.csv')

df['DATE'] = pd.to_datetime(df['DATE'])
df = df.sort_values('DATE')

df.head()
```

	STATION	NAME	LATITUDE	LONGITUDE	ELEVATION	DATE	AWND	AWND_ATTRIBUTES	PGTM	PGTM_ATTRIBUTES	...	WT05	WT05_ATTRIBUTES	WT06	WT06_ATTRIBUTES	WT07	WT07_ATTRIBUTES	WT08	WT08_ATTRIBUTES	WT09	WT09_ATTRIBUTES
0	USW00094846	CHICAGO OHARE INTERNATIONAL AIRPORT, IL US	41.96017	-87.93164	204.8	2016-01-01	14.32	,,W	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3730	USW00094728	NY CITY CENTRAL PARK, NY US	40.77898	-73.96925	42.7	2016-01-01	7.61	,,W	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
7460	USW00093134	LOS ANGELES DOWNTOWN USC, CA US	34.02360	-118.29110	54.6	2016-01-01	2.46	,,W	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1.0	,,W	NaN	NaN
1	USW00094846	CHICAGO OHARE INTERNATIONAL AIRPORT, IL US	41.96017	-87.93164	204.8	2016-01-02	12.08	,,W	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3731	USW00094728	NY CITY CENTRAL PARK, NY US	40.77898	-73.96925	42.7	2016-01-02	6.71	,,W	NaN	NaN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

5 rows × 50 columns

Normalizing city names

```
def get_city(name):
    name = str(name).upper()  # prevents errors
    if 'NY' in name:
        return 'NYC'
    elif 'CHICAGO' in name:
        return 'Chicago'
    elif 'LOS ANGELES' in name:
        return 'LA'
    else:
        return 'Other'

df['CITY'] = df['NAME'].apply(get_city)
```

```
df[['NAME','CITY']].head(10)
```

	NAME	CITY
0	CHICAGO OHARE INTERNATIONAL AIRPORT, IL US	Chicago
3730	NY CITY CENTRAL PARK, NY US	NYC
7460	LOS ANGELES DOWNTOWN USC, CA US	LA
1	CHICAGO OHARE INTERNATIONAL AIRPORT, IL US	Chicago
3731	NY CITY CENTRAL PARK, NY US	NYC
7461	LOS ANGELES DOWNTOWN USC, CA US	LA
2	CHICAGO OHARE INTERNATIONAL AIRPORT, IL US	Chicago
3732	NY CITY CENTRAL PARK, NY US	NYC
7462	LOS ANGELES DOWNTOWN USC, CA US	LA
3	CHICAGO OHARE INTERNATIONAL AIRPORT, IL US	Chicago

Removing unnecessary columns/attributes

```
df = df[['DATE', 'CITY', 'TMAX', 'TMIN', 'PRCP']]
df = df.dropna()
```

```
df['CITY'].value_counts()
```

CITY	count
Chicago	3730
NYC	3730
LA	3728

dtype: int64

Finalizing cleaning & Feature Engineering

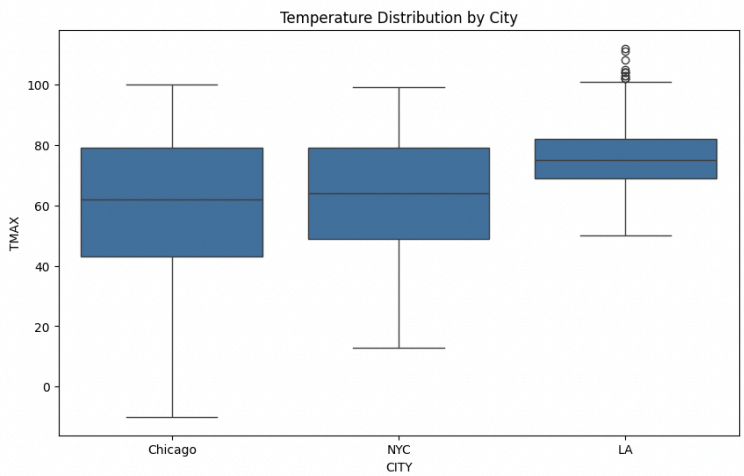
```
# Make a clean copy
clean_df = df.copy()

# Add features (CORRECTLY grouped by city)
clean_df['dayofyear'] = clean_df['DATE'].dt.dayofyear

clean_df['temp_range'] = clean_df['TMAX'] - clean_df['TMIN']

#rolling per city (not global)
clean_df['rolling_mean'] = clean_df.groupby('CITY')['TMAX'].transform(lambda x: x.rolling(7).mean())
clean_df['rolling_std'] = clean_df.groupby('CITY')['TMAX'].transform(lambda x: x.rolling(7).std())

# drop any remaining missing values
clean_df = clean_df.dropna()

# Remove duplicates (important)
clean_df = clean_df.drop_duplicates()

# Reset index
clean_df = clean_df.reset_index(drop=True)

clean_df.head()
```

	DATE	CITY	TMAX	TMIN	PRCP	month	extreme_temp	temp_range	dayofyear	rolling_mean	rolling_std
0	2016-01-07	LA	59.0	45.0	0.30	1	False	14.0	7	62.285714	3.988077
1	2016-01-07	Chicago	41.0	34.0	0.01	1	False	7.0	7	33.285714	4.029652
2	2016-01-07	NYC	46.0	31.0	0.00	1	False	15.0	7	39.857143	5.814596
3	2016-01-08	LA	62.0	44.0	0.00	1	False	18.0	8	62.000000	3.915780
4	2016-01-08	Chicago	42.0	36.0	0.19	1	False	6.0	8	34.857143	5.014265

Additional features were engineered, including day-of-year, temperature range, and rolling statistics (7-day mean and standard deviation). Rolling features were computed separately for each city to preserve temporal consistency.

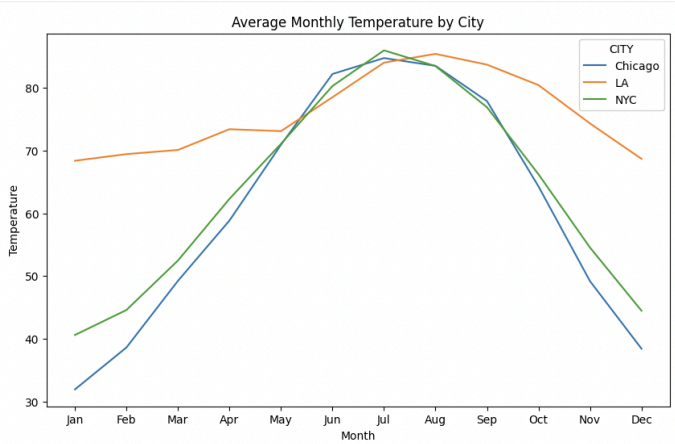
TEMPERATURE DISTRUBTION BY CITY

```python
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
sns.boxplot(data=df, x='CITY', y='TMAX')
plt.title('Temperature Distribution by City')
plt.show()
```



Chicago exhibits the largest variability in temperature, while Los Angeles shows a much tighter distribution, consistent with its Mediterranean climate.

```python
df['month'] = df['DATE'].dt.month


monthly_avg = df.groupby(['CITY','month'])['TMAX'].mean().reset_index()
monthly_avg['month_name'] = monthly_avg['month'].map({
    1: 'Jan', 2: 'Feb', 3: 'Mar', 4: 'Apr',
    5: 'May', 6: 'Jun', 7: 'Jul', 8: 'Aug',
    9: 'Sep', 10: 'Oct', 11: 'Nov', 12: 'Dec'
})

plt.figure(figsize=(10,6))

sns.lineplot(data=monthly_avg, x='month_name', y='TMAX', hue='CITY')

plt.title('Average Monthly Temperature by City')
plt.xlabel('Month')
plt.ylabel('Temperature')

plt.show()
```



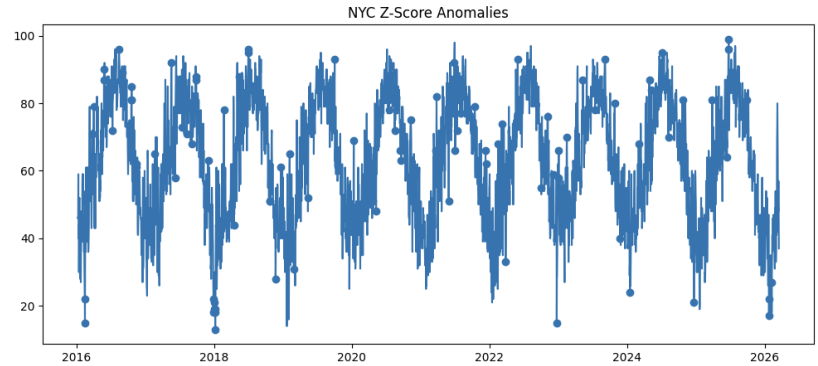
Strong seasonal trends are observed, particularly in Chicago, while Los Angeles remains relatively stable throughout the year.

Z-SCORE MODEL

```
df['seasonal_mean'] = df.groupby(['CITY','dayofyear'])['TMAX'].transform('mean')
df['seasonal_std'] = df.groupby(['CITY','dayofyear'])['TMAX'].transform('std')

df['z_score'] = (df['TMAX'] - df['seasonal_mean']) / df['seasonal_std']

df['anomaly_z'] = abs(df['z_score']) > 2
```

ISOLATION FOREST

```
from sklearn.ensemble import IsolationForest

features = df[['TMAX','PRCP','temp_range','rolling_mean']]

model = IsolationForest(contamination=0.02, random_state=42)

df['anomaly_if'] = model.fit_predict(features)
df['anomaly_if'] = df['anomaly_if'] == -1
```

QUICK MODEL TEST

```
import matplotlib.pyplot as plt

nyc = df[df['CITY'] == 'NYC']

plt.figure(figsize=(12,5))
plt.plot(nyc['DATE'], nyc['TMAX'])

anom = nyc[nyc['anomaly_z']]
plt.scatter(anom['DATE'], anom['TMAX'])

plt.title('NYC Z-Score Anomalies')
plt.show()
```